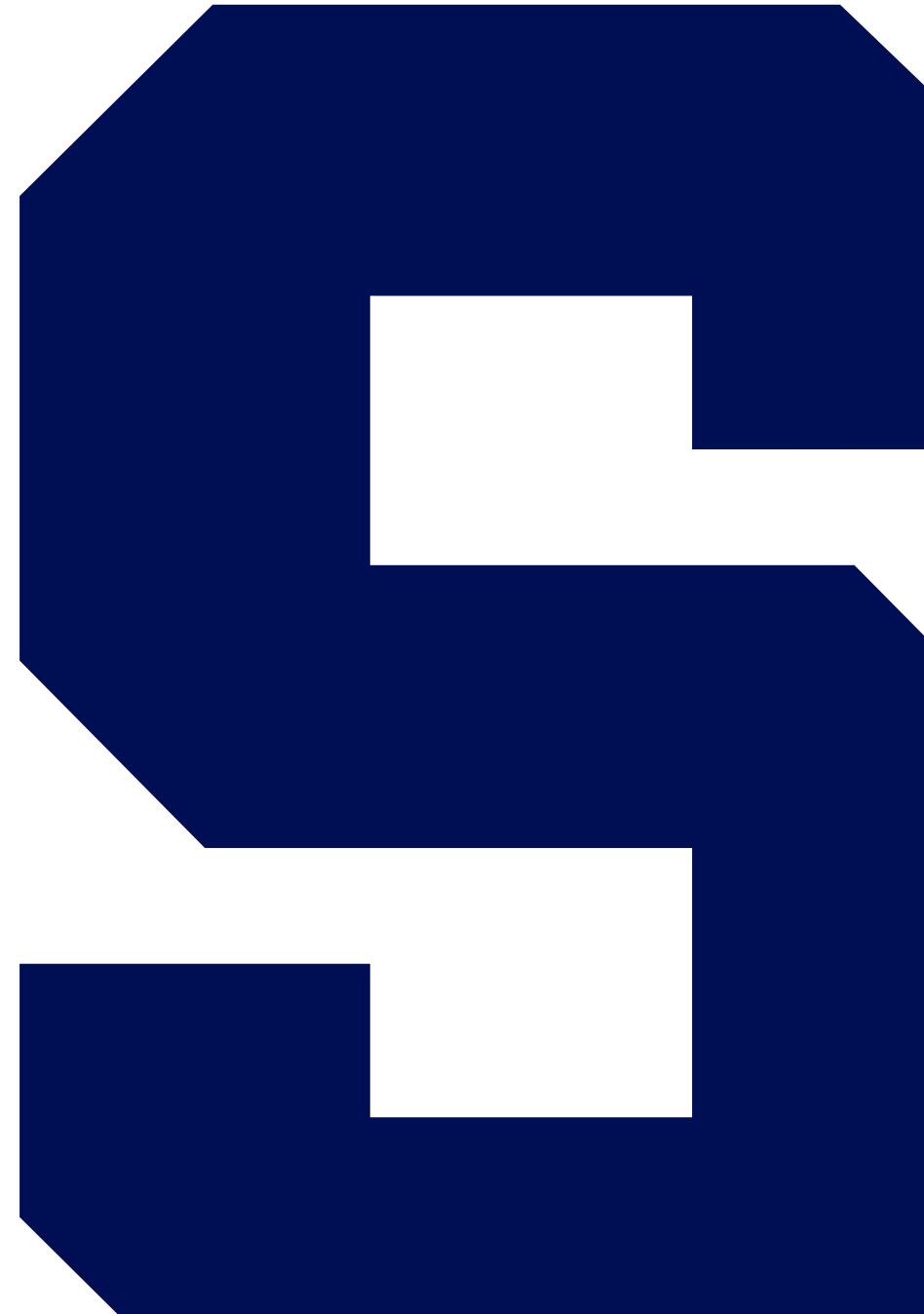
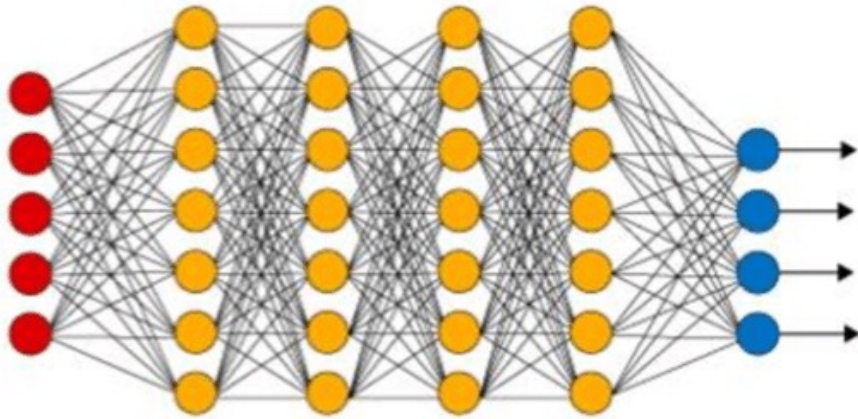


Vanishing and Exploding Gradients



Backpropagation Review

- Backpropagation works by moving from the output layer back to the input layer, carrying the **error gradient** along its path.
- Backpropagation then employs these gradients for updating each parameter through a **gradient descent** step.



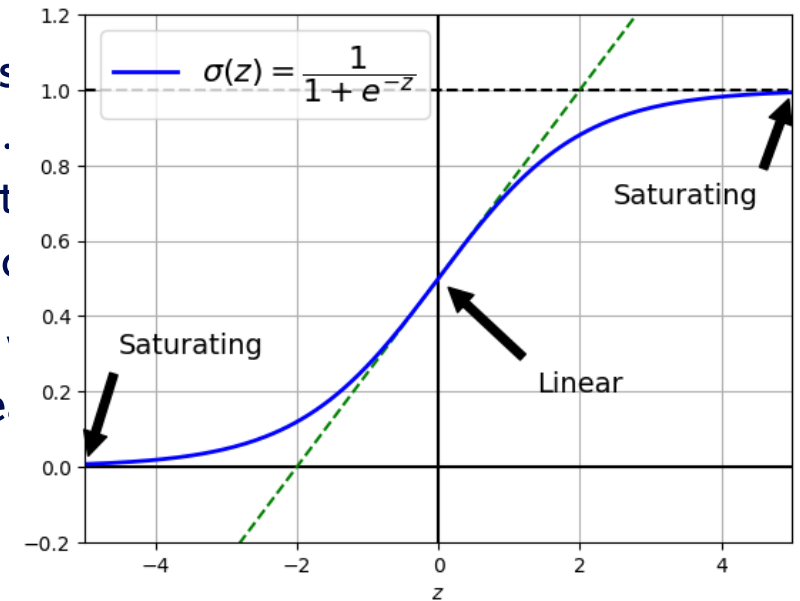
$$w_{new} = w - \alpha * \frac{\delta L}{\delta w}$$
$$b_{new} = b - \alpha * \frac{\delta L}{\delta b}$$

Unstable Gradients

- A problem: gradients often **diminish** as backpropagation approaches the lower (left) layers, or the gradients might sometimes **escalate**, causing excessively large updates to the layer weights and leading the algorithm to diverge.
 - **Vanishing Gradients:** Occur when gradients become very small, effectively preventing the weights from changing their value during training. Common in deep networks with many layers.
 - **Exploding Gradients:** Happen when gradients grow exponentially large, causing large updates to neural network weights during training, often leading to model instability.
-

Key Factors of Unstable Gradients

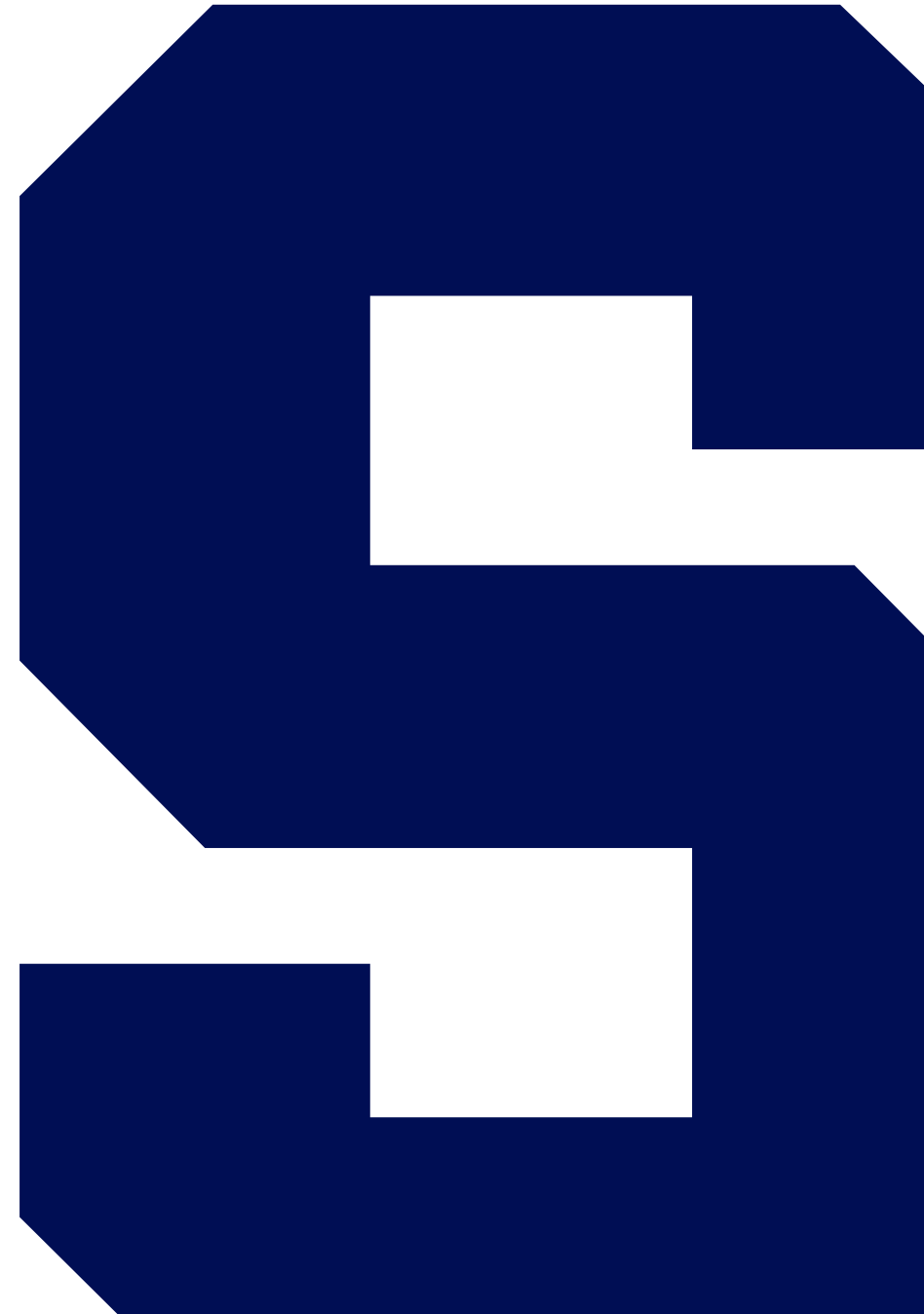
- 1. Activation Function:** The widely used sigmoid (logistic) activation function was problematic. This function saturates at 0 or 1 for large positive or negative inputs, leading to an extremely small derivative in these regions. During backpropagation, these small gradients provided very little information for updating weights in the earlier layers.
- 2. Weight Initialization:** The common practice of initializing weights (mean of 0 and standard deviation of 1) exacerbated the problem. the variance of each layer's outputs was significantly greater than the variance of the inputs. As the network propagated, the more the output variance increased, the more the variance of the weights increased.
- 3. Mean of Sigmoid:** The sigmoid function has a mean of 0.5, not 0, compared to functions like the hyperbolic tangent, which has a mean of 0. This was a problem because it shifted the distribution of the outputs, making it harder to learn the weights in the earlier layers.



Addressing the Problem of Unstable Gradients

1. Better weight/bias initialization strategy to stabilize variance during the forward and backward pass (Glorot uniform initialization).
 2. Adopt the use of better activation functions such as ReLU, to address the saturation issue inherent with the sigmoid function.
 3. Normalize the inputs to hidden layers through re-centering and re-scaling.
-

Activation Functions

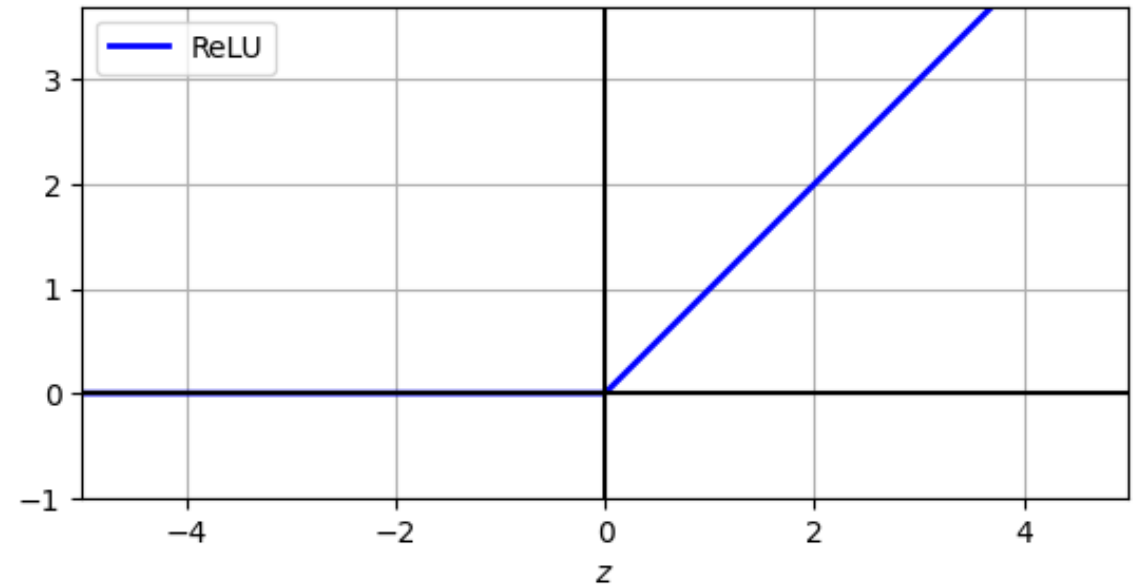


The Importance of Activation Functions

- Unstable gradients in deep neural networks were partially due to the choice of activation function.
 - While sigmoid activation functions have been long popular, other functions, particularly the ReLU (Rectified Linear Unit), are more effective in deep learning.
 - Many effective activation functions have been developed.
-

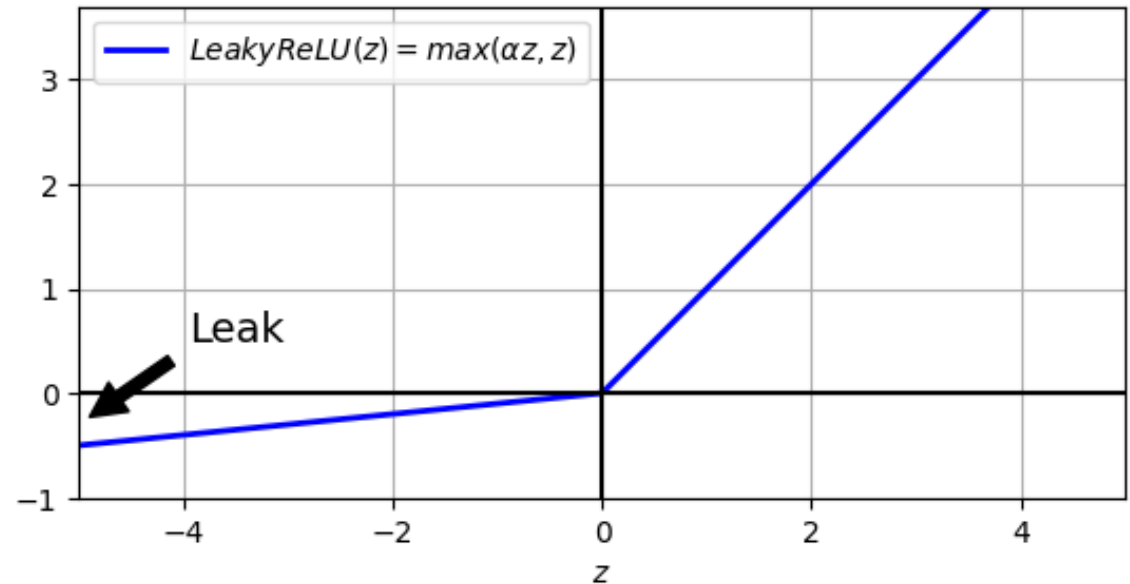
Rectified Linear Unit (ReLU)

- ReLU is advantageous because it does not saturate for positive values and is computationally efficient.
- However, it can suffer from the "dying ReLUs" problem, where neurons become inactive and only output zero.
- Occurs when a neuron's weights adjust so that its input is negative for all training instances, leading to zero output and no gradient updates.
- This issue is especially prevalent with large learning rates, potentially resulting in a significant portion of the network becoming inactive.



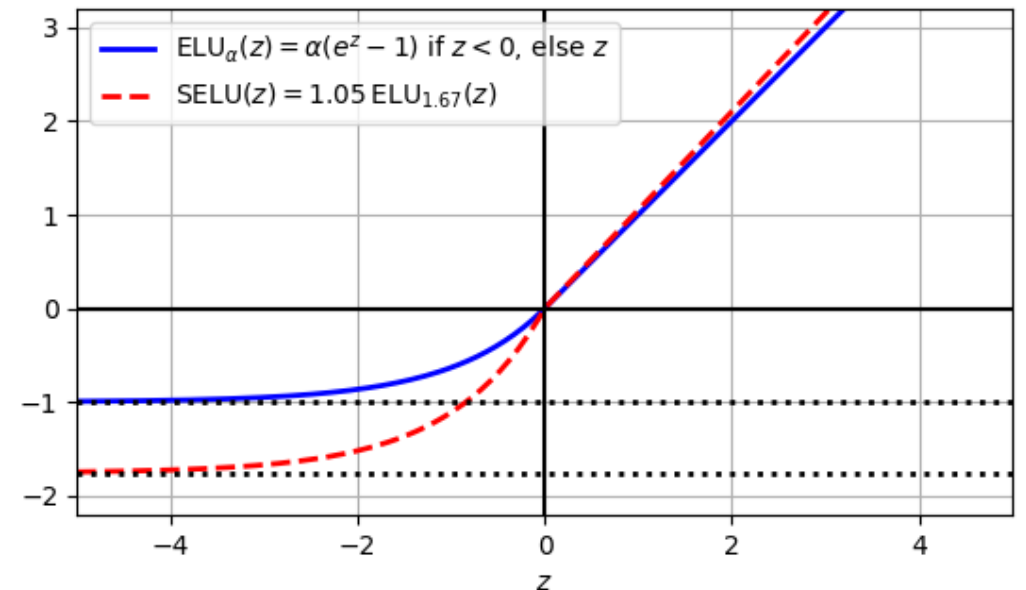
Leaky ReLU

- The leaky ReLU has a hyperparameter, α , a small, positive slope for $z < 0$.
- Ensures that leaky ReLUs maintain some activity, preventing them from “dying.”
- Research has shown that leaky ReLUs generally outperform the standard ReLU.
- A variant, randomized leaky ReLU (RRReLU), which varies α in a range during training, also shows good performance.
- Parametric leaky ReLU (PReLU), allows α to be learned during training. This has shown superior performance on large image datasets but might overfit on smaller datasets.



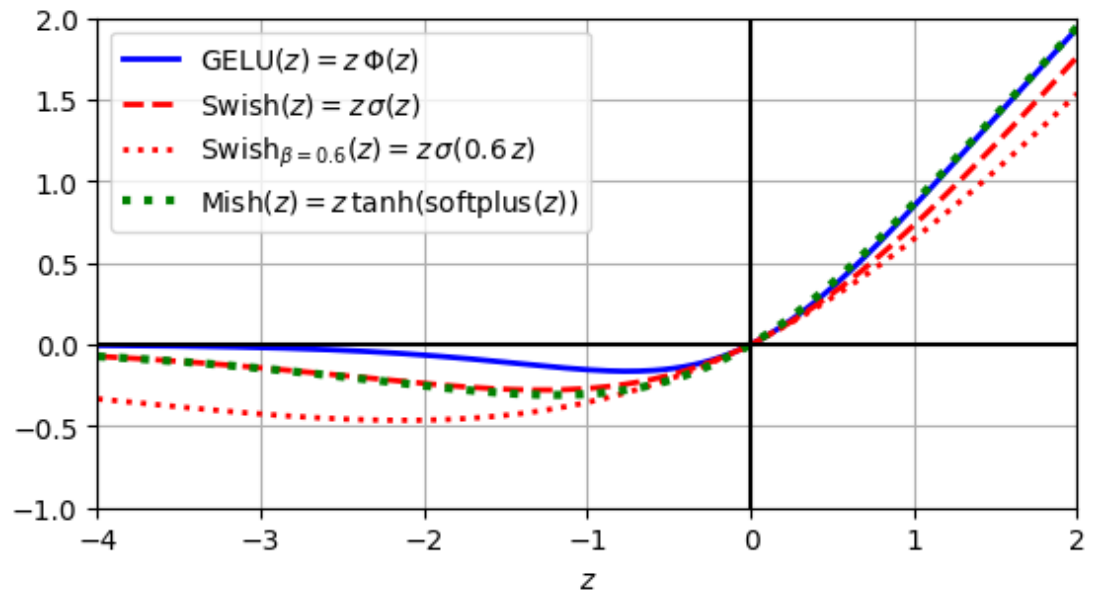
ELU and SELU

- ReLU and its variants are not smooth functions (they are discontinuous at $z = 0$), sometimes causing issues for gradient descent.
- Exponential Linear Unit (ELU) and Scaled Exponential Linear Unit (SELU) are smooth variants of ReLU.
- Each address the “dying ReLU” problem.
- Computationally more expensive but can converge faster.
- SELU is self-normalizing (under specific conditions), automatically tending toward zero mean and unit variance.
- Can lead to faster convergence overall.



GELU, Swish, and Mish

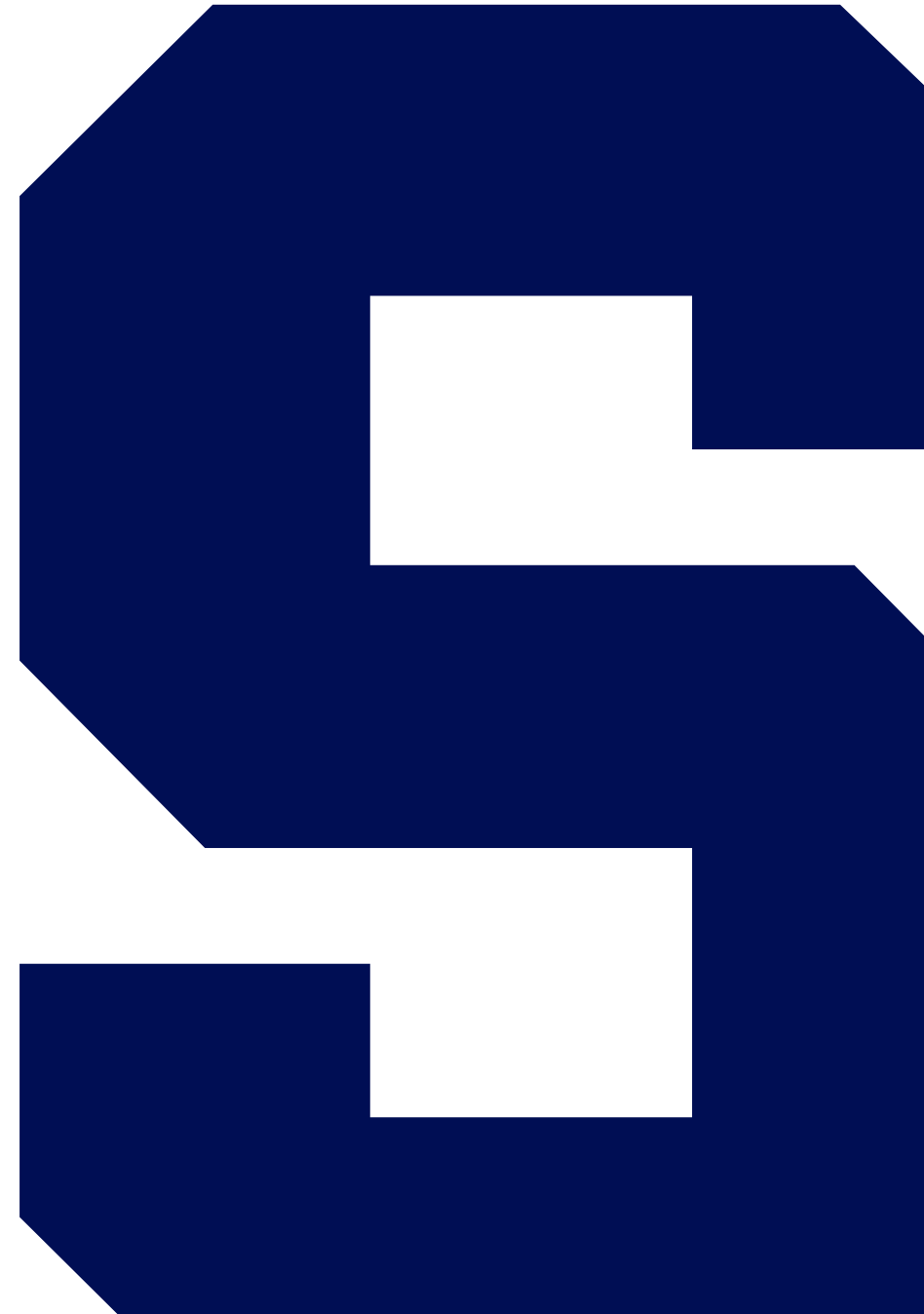
- Gaussian Error Linear Unit (GELU), Swish, and Mish generally perform better on most tasks than ELU and SELU.
- GELU is especially popular with transformer models but is computationally more expensive than standard ReLU.
- Swish was developed at Google and has a self-gating property, which allows it to conditionally control the flow of information.
- Mish is a newer variant and has been shown to outperform many activation functions but is computationally expensive.



Activation Function Best Practices

- Which activation function should you choose for the hidden layers of your network?
 - ReLU is still a good default.
 - ReLU often performs just as well as more sophisticated activation functions for simple tasks, with the benefit of being faster to compute.
 - Swish is a good default for more complex tasks.
 - Mish is sometimes slightly better than Swish, but it requires more compute.
 - If you have spare time and computing power, you can use cross-validation to evaluate different activation functions.
-

Normalization



Batch Normalization

- Another technique that addresses the problem of exploding/vanishing gradients is **batch normalization**.
 - Batch normalization improves the speed, performance, and stability of ANNs by addressing the issue of **internal covariate shift**.
 - Internal covariate shift is where the distribution of each layer's inputs changes during training, making training slower and more complicated.
 - Batch normalization may also help deep networks to generalize better.
-

Batch Normalization Process

Given a mini-batch of data (the set of training examples in each batch, usually 32), the algorithm normalizes the activations of each layer's inputs for each batch. The formal steps are as follows:

1. Normalize the batch using the mean and variance calculated for each feature across the mini-batch.

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

2. Apply learnable scale and shift parameters to each normalized feature. This helps to restore the representational capacity of the network that may have been lost during the normalization step.

$$y_i = \gamma \hat{x}_i + \beta$$

Performance and Role in Regularization

- **Training Speed:** Batch normalization can accelerate the training of deep networks by allowing the use of higher learning rates and reducing the sensitivity to weight initialization.
 - **Regularization:** While not a substitute for dropout, batch normalization provides a mild form of regularization, often reducing the need for dropout or other regularization techniques.
 - **Effectiveness:** Batch normalization has been shown to be highly effective for deep networks and is widely used in state-of-the-art architectures.
-